

《信息系统安全》第十二讲

操作系统安全

Android安全机制

2023年12月4日

周亚建

yajian@bupt.edu.cn

Overview of Last Class

**Windows
资源管理
方式**

- Workgroup
- Domain

**活动目录
的结构**

- 逻辑结构
- 物理结构

组策略

未来安卓系统必然会跌下神坛并消失！

可能很多人不相信，也不会相信安卓拥有全球第一大用户，怎么可能会跌下神坛？靠华为的鸿蒙OS打下去吗？



Google正在做一件事——从头开始研发了一款全新zircon内核的Fuchsia OS系统

为什么要从头开始呢？

谷歌认为未来的系统是只有“一个”，大到PC、平板、手机，小到ATM和GPS单元的全领域系统，巧的是华为鸿蒙OS的理念也是如此，目前的安卓系统显然已经无法满足这一需求，所以未来必然遭到淘汰。

未来新型系统Fuchsia OS系统和鸿蒙OS都有一个特点是可以做到无缝衔接安卓应用程序，至此，你还认为安卓系统有未来吗？

Fuchsia的使命



任务一

解决Android碎片化严重的问题，这次从一开始就用最合理的底层方案，如果Fuchsia OS成功了，那将是个类似苹果一样强悍的生态系统，重点是所有底层和最高的权限都在谷歌自己手里，想干嘛就干嘛，而Android则不行，有些东西还是受限于人。



任务二

手机系统必须与PC系统无缝衔接，这样才可以保持生态的互动性。苹果电脑、iPad、手机，这三大件在办公领域可以发挥超强的互动能力，大大提升办公效率，这也是很多商务人士选择苹果产品的主要原因之一。



任务三

Fuchsia OS一定会全面兼容Android应用，否则一个全新的系统出来，没有可以用的软件，基本一出生就进了棺材，兼容性这一点根本不用猜，100%会支持。



任务四

提升谷歌整体盈利预期。要知道比起硬件盈利，软件上的盈利更凶猛，而且成本更低，赚钱更容易。但这一切都是建立在，你有自己完整生态的基础上，所以无论如何谷歌都会拿下这一块蛋糕。



任务五

统治所有智能硬件。这件事与苹果和小米正在做的事很像，我们以小米为例。米家APP可以控制所有米家生态企业售卖的产品，这就意味着小米的智能生态是成功的，因为没有人愿意手机里装十几个APP来控制。苹果目前也是如此，很多智能家居产品全都可以通过苹果产品直接控制。

未来安卓系统必然会跌下神坛并消失！



HarmonyOS 2.0
连接无限可能

巧了！鸿蒙OS思路似乎又撞车了，不同的是华为鸿蒙已经在自家硬件设备上实现了初级的多屏互动体验...

HarmonyOS

分布式能力造就新硬件、新交互、新服务，打开焕然一新的全场景世界。

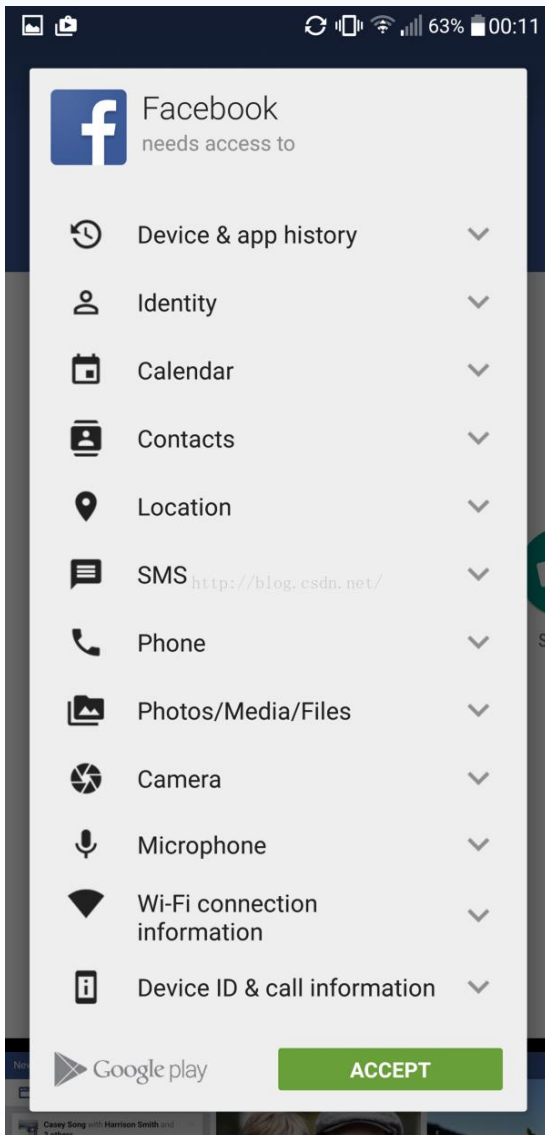
华为要研发鸿蒙OS还有一个更重要的原因是，安卓系统没有可能达到iOS那种流畅度，华为尽管研发出来方舟编译器解决部分应用问题，但要完美解决这一越用越卡诟病，只能从内核一步一步完善，所以鸿蒙OS似乎成为了一个必然的产物。

未来所有硬件平台只有一个“系统”

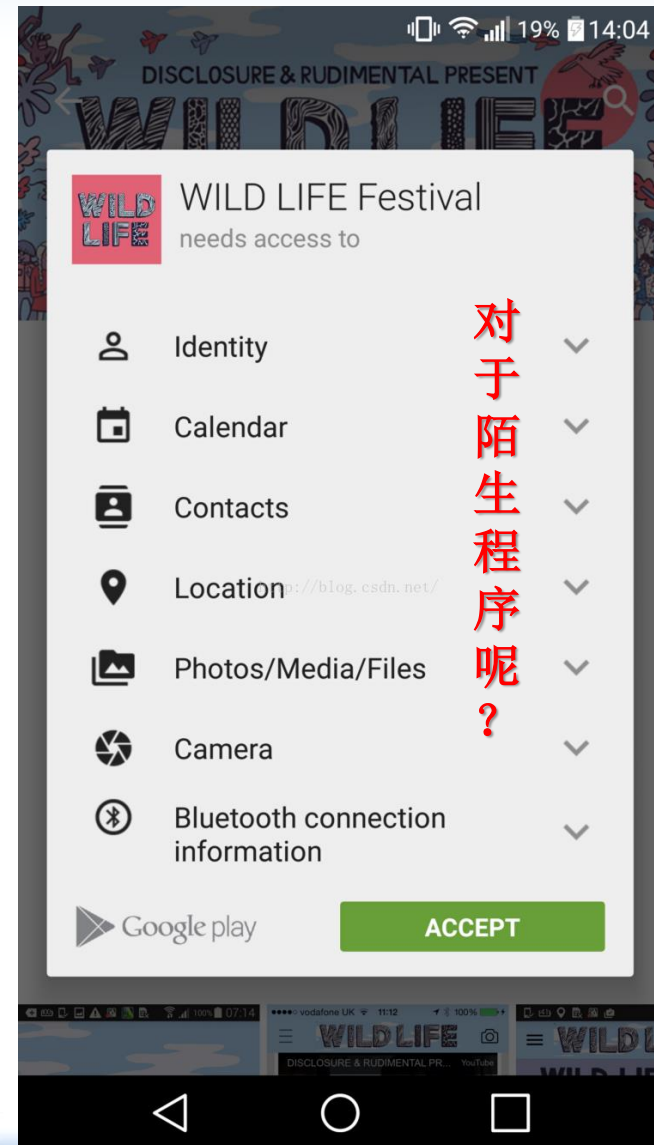


鸿蒙OS的发展告诉我们未来所有硬件平台只有一个“系统”，完美的系统之间的无缝衔接才能有极致无缝的优质体验。

这一讲要解决什么问题？



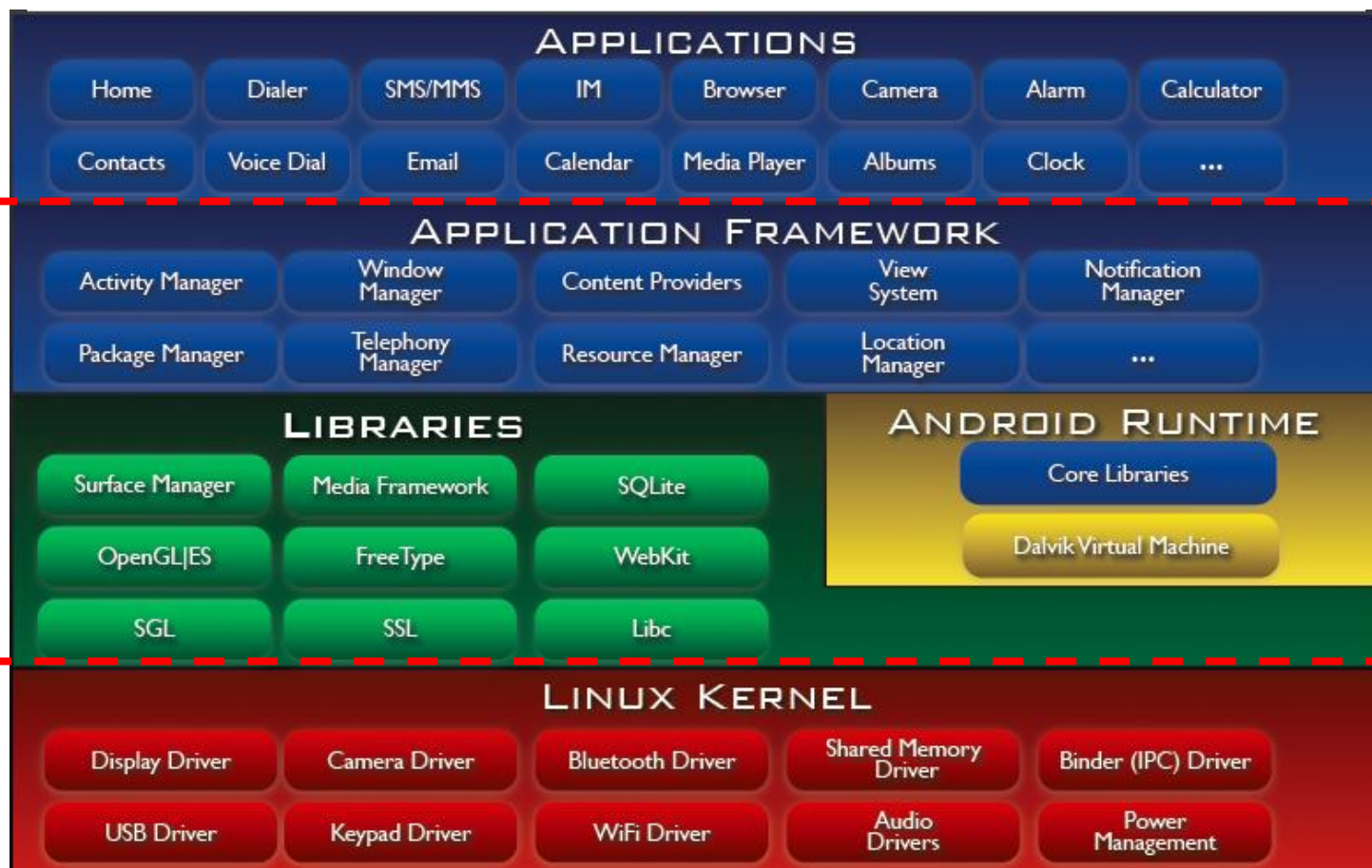
Facebook权限请求列表堪称巨大，但是**Facebook**已经和它的用户们建立了良好的信任关系，所以用户在安装的时候，会忽略那一大串的权限请求



**Android
系统架构**



Android Architecture



第4层：应用程序

第3层：应用程序
框架

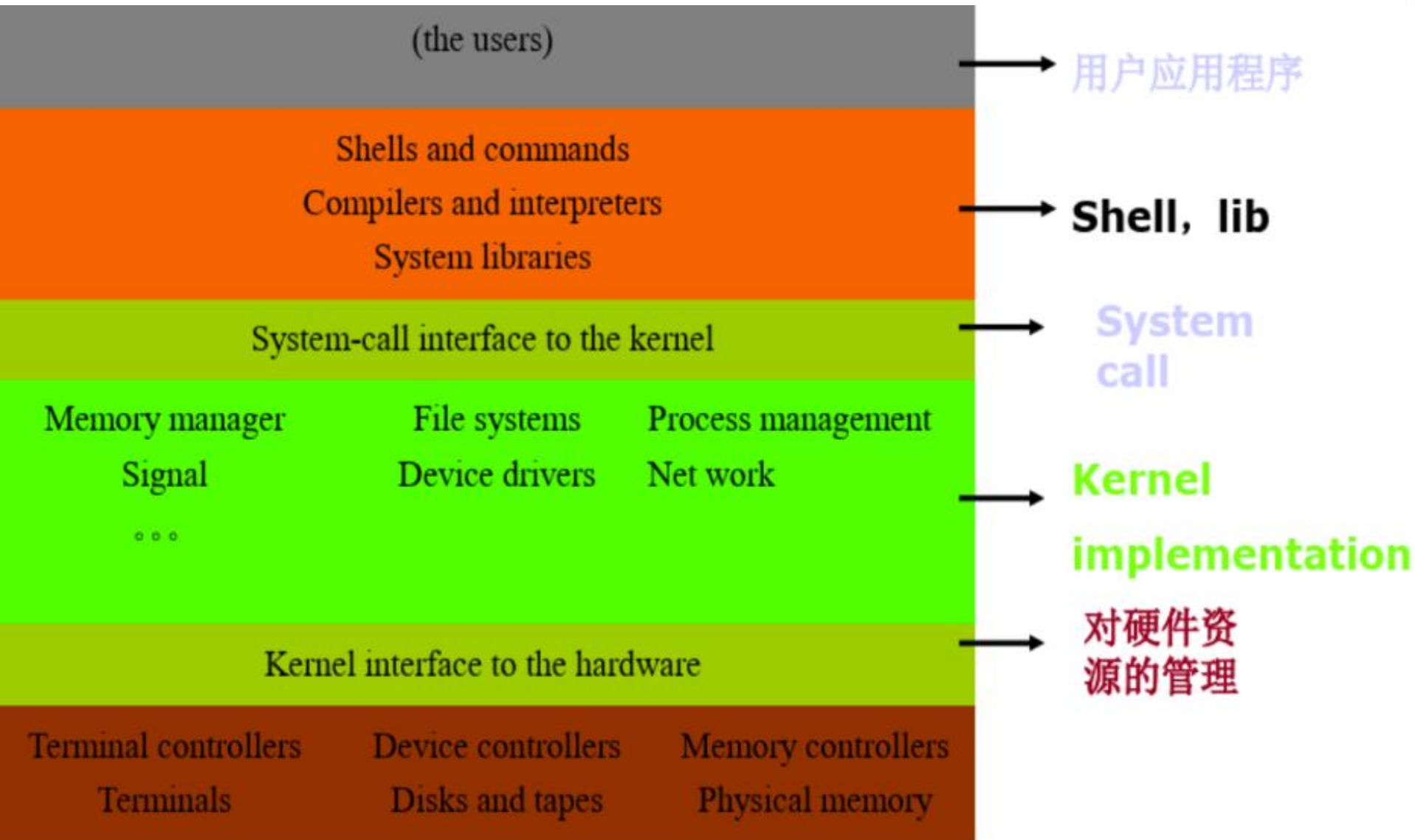
第2层：本地类库及
Java运行环境

第1层：Linux内核及
驱动

Android本身是一套软件堆叠（Software Stack），或称为“软件叠层架构”，叠层主要分成三层：操作系统、中间件、应用程序。

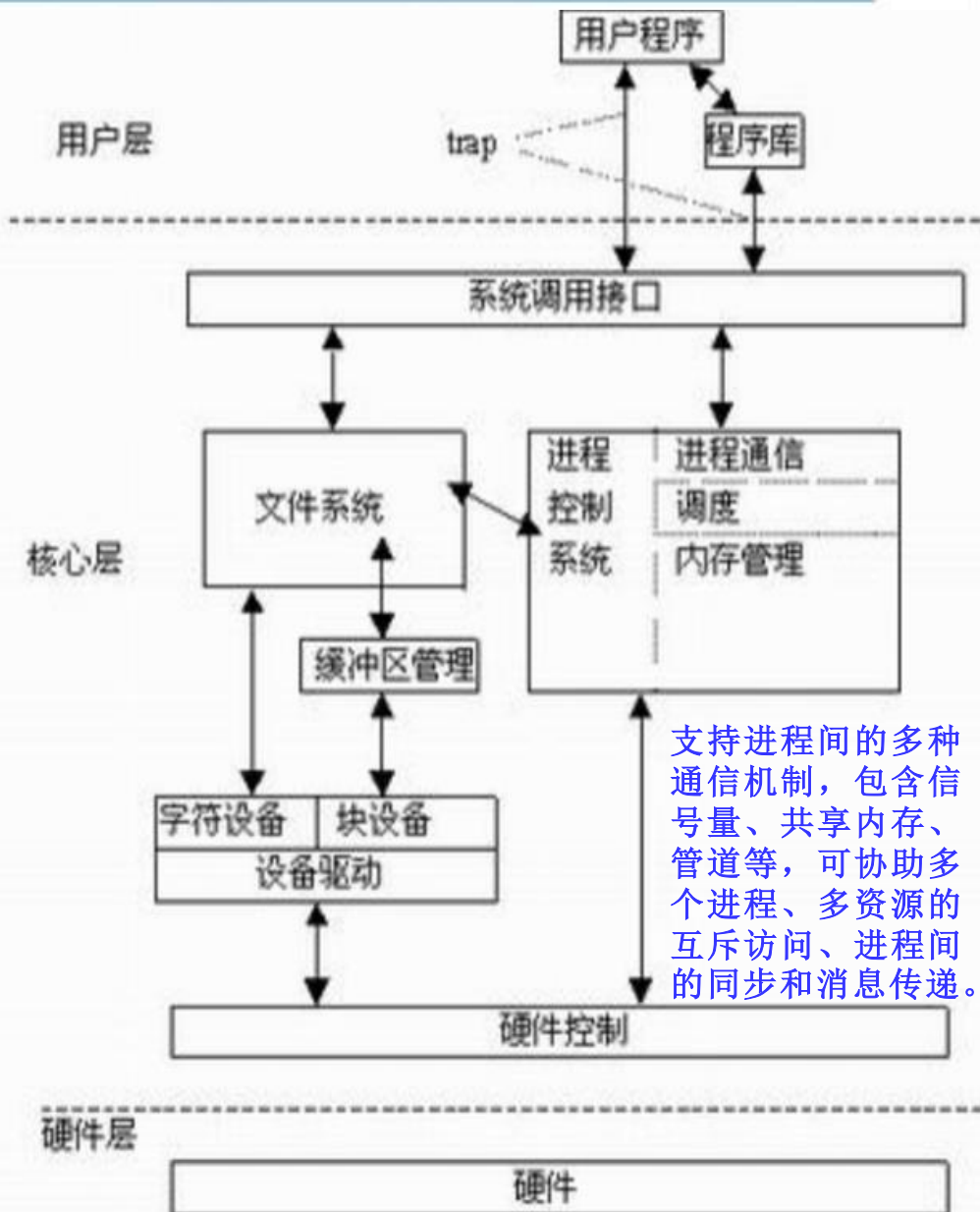
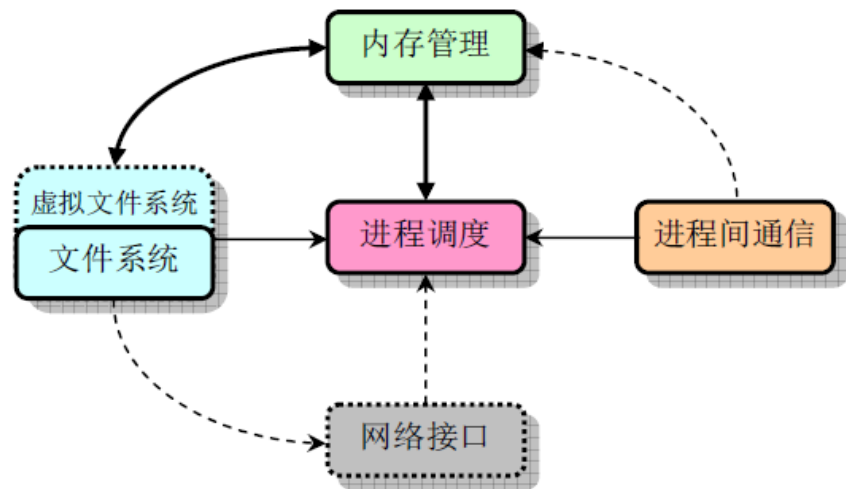


Linux操作系统的典型架构

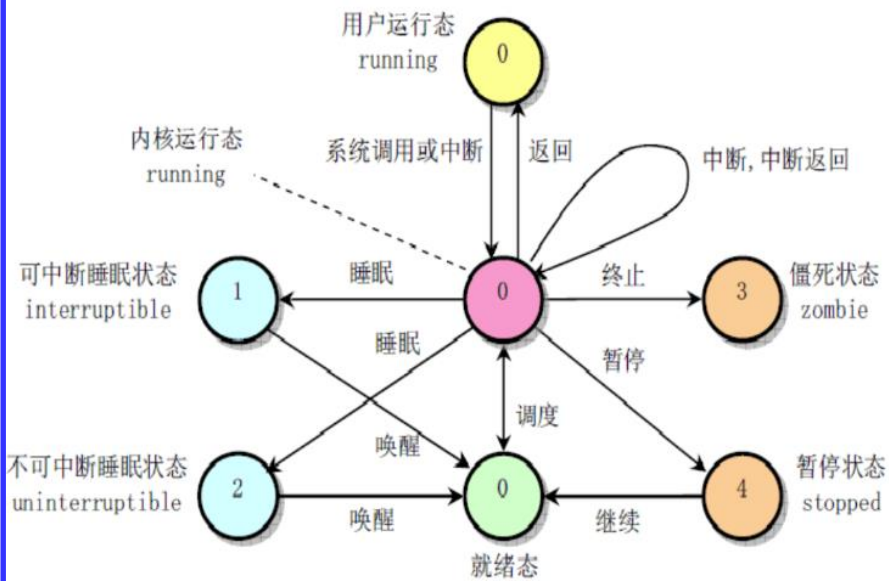




Linux操作系统的内核

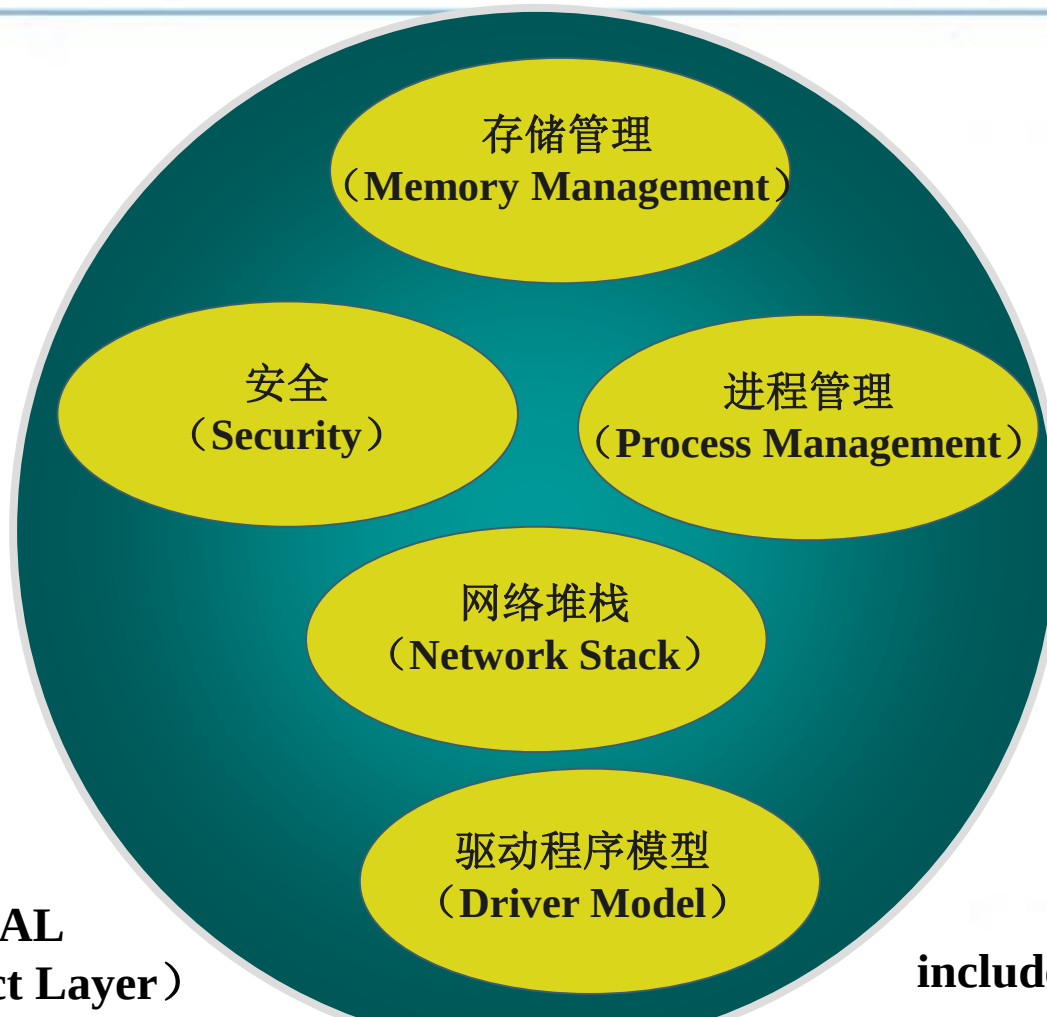


进程调度





Android架构: Linux内核及驱动



硬件抽象层HAL
(Hardware Abstract Layer)

include/linux/device.h





Android架构: Linux内核及驱动

- Android 虽然运行于Linux kernel之上，但并不是GNU/Linux 因为在一般GNU/Linux 里支持的功能，Android 大都没有支持，包括Cairo、X11、Alsa、FFmpeg、GTK、Pango及Glibc等都被移除掉。

Android以Bionic取代Glibc、以Skia取代Cairo、再以opencore取代FFmpeg等等。

Android系统在系统启动初始化与编程接口方面不同于标准的Linux 系统。

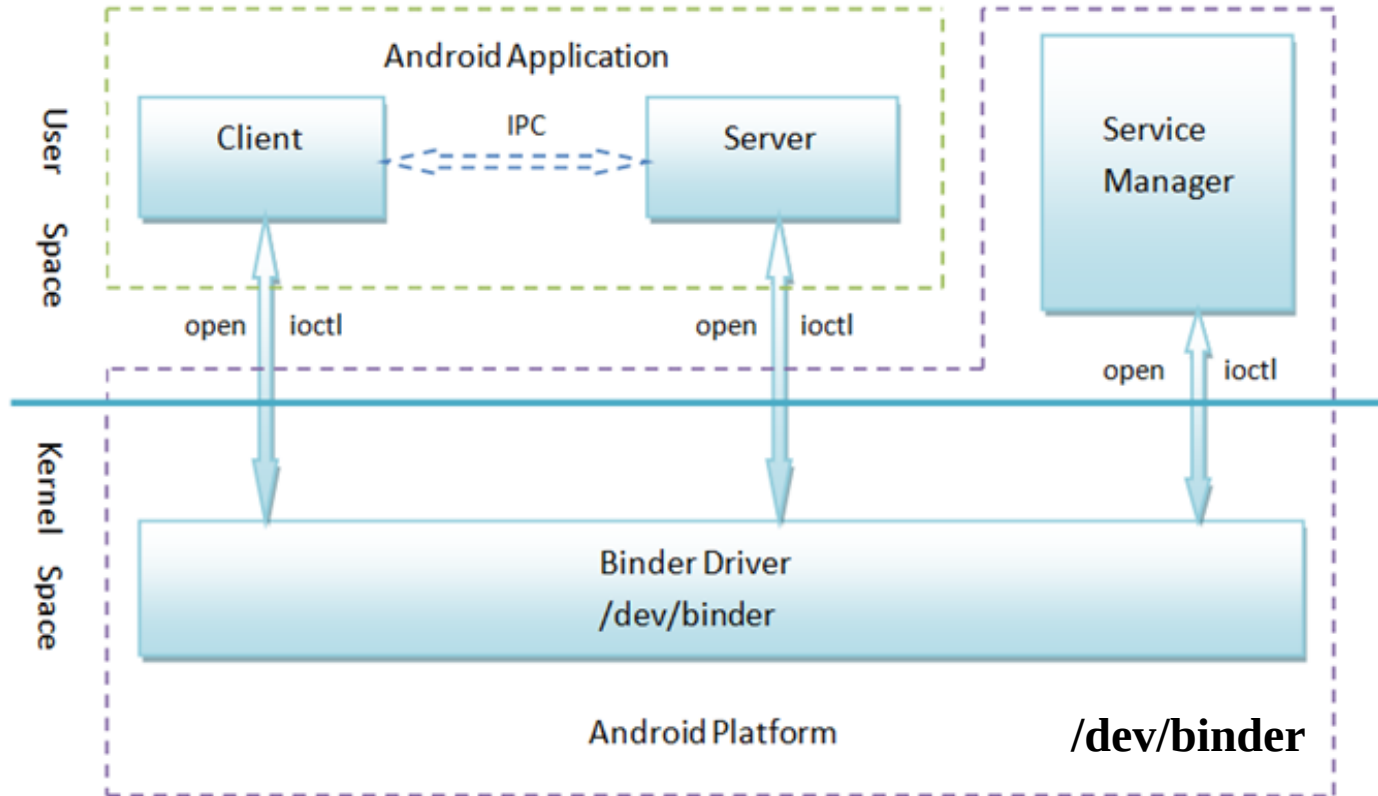




Android架构：Linux内核及驱动

使用Binder作为进程间通信机制，取代传统进程间通信机制

Linux已经拥有的进程间通信IPC（Internet Process Connection）手段包括：管道（Pipe）、信号（Signal）和跟踪（Trace）、插口（Socket）、报文队列（Message）、共享内存（Share Memory）和信号量（Semaphore）。





Why Linux?

- Great memory and process management
- Permissions-based security model
- Proven driver model
- Support for shared libraries
- It's already open source!

内核优化

为了支持 Android，Android 的主线内核做了相应优化，主要体现在以下几个方面：

- Alarm
- Ashmem
- Binder
- Power Management
- Low Memory Killer
- Kernel Debugger
- Logger



Linux内核存在的安全问题

在内核层，进程往往以较高权限运行，其漏洞可能导致高级权限泄露

早在2010年，美国软件分析公司Coverity就曾宣布Android内核中存在大量漏洞，其中包括88个高风险漏洞，有可能会用户敏感信息泄露

刘剑等人开发了一套基于控制流挖掘的Android系统代码漏洞分析工具，针对Android内核代码的多种典型错误构建相关的分析脚本进行了分析，从而检测发现了代码库中一系列脆弱点。

CVE-2012-0056

产生原因是Linux不能正确完成/proc/<pid>/mem的权限控制，导致攻击者通过改写一个特权进程的内存，可以获取root权限来执行任意的root权限的代码，该漏洞被用于Android 4.0版本的本地提权。

CVE-2013-6282

ARM v6/v7架构的Linux内核中的get-user/put_user接口没有验证目标地址，由于硬件架构的更迭，get-user/put_user最初用于实现和控制域切换的功能被弃用了，导致任何使用该API的内核代码都可能存在安全隐患，让任意应用来读写内核内存，造成权限泄露。



Linux内核存在的安全问题

Binder作为Linux内核层的进程通信机制，为进程间的共谋攻击提供便利





如何增强Linux内核的安全性

2010年，Shabtai等人提出了以SELinux为内核实现Android安全机制增强的思路

2011年，Bugiel等人以TOMOYO Linux为安全增强内核，开发了TrustDroid系统

2013年，Smalley等人实现了基于SELinux内核的Android系统

由DAC
转变为
MAC

都是从Linux层面改变
文件的访问控制

SELinux 是 2.6 版本的 Linux 内核中提供的强制访问控制（MAC）系统，默认安装在 Fedora 和 Red Hat Enterprise Linux（RHEL）上



Android架构：本地类库及Java运行环境

- 一系列的C/C++库，相当于Android系统在Linux核心系统上的功能扩展，为Application Framework层的许多功能提供支持，通过JNI供Java调用。

Android中，经常会用到.so文件，.so文件是一个C/C++的函数库，Android中调用.so文件都是通过JNI的方式。

Native Libraries





Android架构：本地类库及Java运行环境

Android 包含一些C/C++库，这些库能被 Android 系统中不同的组件使用。它们通过 Android 应用程序框架为开发者提供服务。

系统 C 库	一个从 BSD 继承来的标准 C 系统函数库（libc），它是专门为基于 embedded Linux 的设备定制的。
媒体库	基于 PacketVideo OpenCORE；该库支持多种常用的音频、视频格式回放和录制，同时支持静态图像文件。编码格式包括MPEG4、H.264、MP3、AAC、AMR、JPG、PNG
Surface Manager	对显示子系统的管理，并且为多个应用程序提供了2D 和 3D 图层的无缝融合
LibWebCore	一个最新的Web 浏览器引擎，支持Android 浏览器和一个可嵌入的Web 视图
SGL	底层的 2D 图形引擎
3D libraries	基于 OpenGL ES 1.0 APIs 实现，该库可以使用硬件 3D加速（如果可用）或者使用高度优化的3D 软加速
FreeType	字体引擎：位图（bitmap）和矢量（vector）字体显示
SQLite	一个对于所有应用程序可用、功能强劲的轻型关系型数据库引擎

Native Libraries



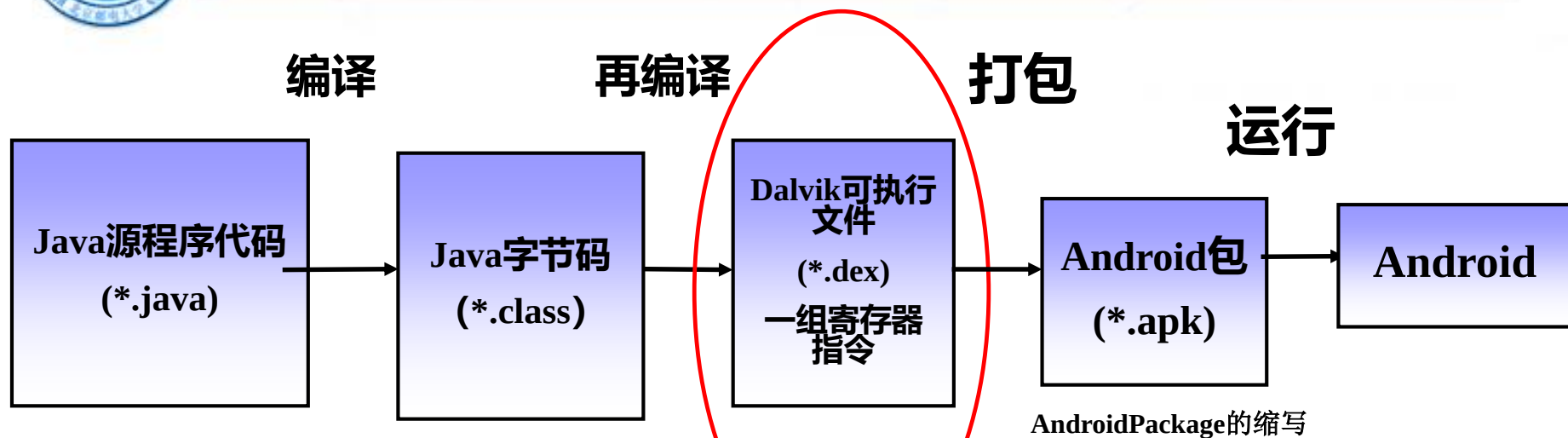
Android架构：本地类库及Java运行环境

应用程序的运行空间，包括一个实现Java语言大多数功能的核心库，以及一个类似于Java虚拟机的Dalvik虚拟机，所有应用程序都是在虚拟机中运行。





Dalvik虚拟机 (DVM)



- Java文件----应用程序源文件
 1. Android 本身相当一部分都是用Java 编写而成
 2. Android 的应用使用Java 来开发
- Class文件----Java编译后的目标文件
 1. Google使用Dalvik 来运行应用程序
 2. Android的class 文件是编译过程中的中间目标文件，需要链接成dex 文件才能在Dalvik 上运行。



Dalvik虚拟机（DVM）

- Dalvik虚拟机（**D**alvik **V**irtual **M**achine）是运行Dalvik 可执行文件（*.dex）的虚拟计算机系统；
- 使用Java语言编写的Android程序，实际上是运行在DVM之上，而不是运行在Linux操作系统上；



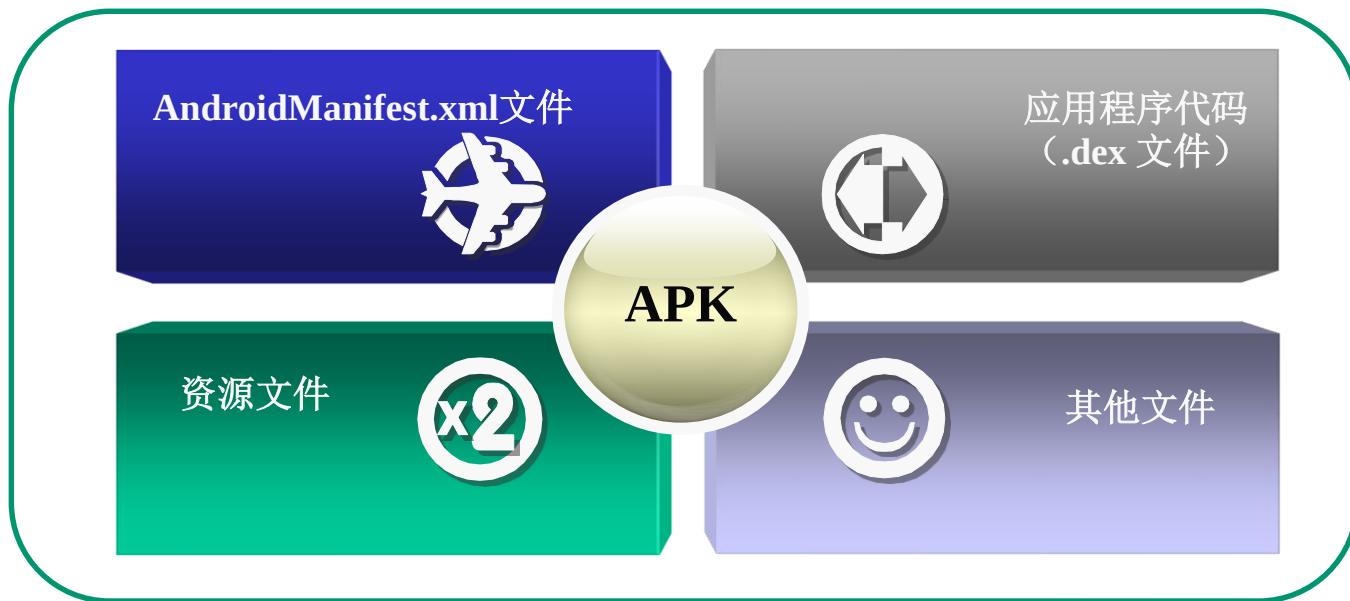
Dalvik VM 针对手机程序/CPU 做过最佳化，可以同时执行许多VM 而不会占用太多资源。



apk文件

● apk文件----Android上的安装文件

1. apk 是Android 安装包的扩展名，一个Android 安装包包含了与该Android 应用程序相关的所有文件



2. 一个工程只能打进一个.apk文件。
3. apk 文件的本质是一个zip包。



apk文件

~/Android/packages/experimental/HelloAndroid

----AndroidManifest.xml

配置文件

----Android.mk 编译脚本文件

----src 源代码文件

----shy/luo/hello

----HelloAndroid.java

----res 资源文件

----layout

----main.xml

----values

----strings.xml

----drawable

----icon.png

mk文件

可隔离CPU、平台、ABI等差异，开发人员只需要简单修改mk文件（指出“哪些文件需要编译”、“编译特性要求”等），就可以创建出so。它可以自动地将so和Java应用一起打包，极大地减轻了开发人员的打包工作。



本地类库及Java运行环境存在的安全问题

Android系统在Linux内核之上实现了自己的系统架构层，一旦该层出现安全问题，即有可能导致Root权限的泄露

CVE-2010-1119

可能对基于系统类库的所有应用产生严重的安全威胁

- Aggregations, pivots and patterns

CVE is a list of entries—each containing an identification number, a description, and at least one public reference—for publicly known cybersecurity vulnerabilities.

CVE Entries are used in numerous cybersecurity products and services from around the world, including the U.S. National Vulnerability Database (NVD).

Common Vulnerabilities and Exposures (CVE)



Android架构: **Application Framework**

应用框架层是Android应用开发的基础，开发人员大部分情况是在和它打交道。

APPLICATION FRAMEWORK

Activity Manager

Window Manager

Content Providers

View System

Notification Manager

Package Manager

Telephony Manager

Resource Manager

Location Manager

...

应用程序框架层包括活动管理器、窗口管理器、内容提供者、视图系统、包管理器、电话管理器、资源管理器、位置管理器、通知管理器和XMPP服务十个部分。



- 开发人员可以完全访问核心应用程序所使用的API框架
- 任何一个应用程序都可以发布自身的功能模块，而其他应用程序则可以使用这些已发布的功能模块。基于这样的重用机制，用户就可以方便地替换平台本身的各种应用程序组件。



Android架构: Application Framework

Android 应用框架中的各个模块都可以被复用，各种服务也可以被复用，开发人员可以更轻松地开发优秀的Android应用。

开发过程中常用到的基本框架组件

丰富而又可扩展的视图
(View System)

可以用来构建应用程序，包括列表 (lists)、网格 (grids)、文本框 (text boxes)、按钮 (buttons)，甚至可嵌入的Web 浏览器

内容提供器
(Content Providers)

使得应用程序可以访问另一个应用程序的数据 (如联系人数据库)，或者共享它们自己的数据

资源管理器
(Resource Manager)

提供非代码资源的访问，如本地字符串、图形、和布局文件 (layout files)

通知管理器
(Notification Manager)

使得应用程序可以在状态栏中显示自定义的提示信息

活动管理器
(Activity Manager)

管理应用程序生命周期并提供常用的导航回退功能





系统架构层的安全问题

系统架构层的安全威胁同样往往来自于安全漏洞

Android Zygote漏洞是Android 2.3版本之前存在的漏洞，Dalvik虚拟机对应用程序的执行过程审核不严格，造成应用程序可以不断地创建进程，当耗尽所有的PID时，PID循环使用0，导致Root权限的泄露

CVE-2010-1119漏洞是一个WebKit库的Use-after-free类型的漏洞，受影响的系统包括Android 2.0至2.1.1操作系统的浏览器，该漏洞可在受影响的系统中执行任意代码或者造成DoS攻击，可以读取手机中的SMS短信数据或者其他敏感数据

应用在调用API（特别是安全相关的API）过程中的不规范操作，往往会带来更多的安全后果。Fahl等人在研究了13500个热门的免费应用后，发现其中有1074个应用可能存在SSL/TLS没有被正常使用，并且可能遭受中间人攻击，在对1100个应用进行了手动测试后，发现了41个应用被成功实现了中间人攻击。Egele等人对Google Play市场上的11748个应用进行了测试，发现有10327个应用使用了加密API并在使用过程中犯下了至少一个错误



系统架构层的安全增强

将SELinux引入系统内核，为系统架构层提供更加安全的服务，实现更加安全的策略

Bugiel等人在以SELinux为基础的Android 4.0系统架构层指定了一套协议语言，提供了定制安全策略的服务，从而使SELinux的安全特性在Android系统的架构层得以应用

完善Android现有的安全机制、细化安全策略

Android系统的3大安全机制——签名机制、权限机制和沙盒机制中，签名机制和沙盒机制设计较为良好，只是偶尔爆出安全漏洞，较少得到安全研究者的关注；但权限机制因其“全是或全否”的粗粒度管理方式备受人们的诟病，而且应用安装后的行为得不到任何监控，对于应用运行过程的行为监控基本为零

API安全实现和规范使用



Android架构: Application

- Android平台不仅仅是操作系统，也包含了许多应用程序，包括客户端、SMS短消息程序、日历、地图、浏览器、联系人管理程序等。

所有的应用程序都是使用JAVA语言编写的，并且这些应用程序都可以被开发人员开发的其他应用程序所替换，这点不同于其他手机操作系统固化在系统内部的系统软件，更加灵活和个性化。





Android系统一个非常重要的特性

- 在Android中，一个应用程序可以使用其它应用程序的组件，这是Android系统一个非常重要的特性

在需要的时候Android会启动另外一个程序的部分代码，这部分代码实现了用户请求执行的功能，而不是在用户的应用程序中直接包含另外一个应用程序的代码。

例如，编写的应用程序需要显示一个可以滚动的图片列表，如果其他某应用程序已经开发了具有此功能的组件并对外发布了，程序员可直接调用这个组件来显示图片，而无需重新开发一个具有此功能的组件。



Android系统一个非常重要的特性

为了实现这样的功能，**Android**必须能够在其他应用程序发出请求的时候启动一个应用程序进程并实例化部分**Java**对象

因此与其他大部分系统不同，运行在**Android**系统上的应用程序并没有一个应用程序入口点（类似**Java**程序中的**main**方法）。

相反，**Android**应用程序包含一些运行应用程序所必须的、能够被**Android**系统实例化的组件。



Android的组件模型



服务是运行在后台的功能模块，如文件下载、音乐播放程序等

Activity 是为用户操作而展示的可视化用户界面

专注于接收广播通知信息，并做出对应处理的组件

Android 平台应用程序间数据共享的一种标准接口，它以类似于 URI（Universal Resources Identification）的方式来表示数据



Android的组件模型

- 需要注意的是，并不是每个Android应用程序都必须构建这4个组件，有些可能由这些组件的组合而成。



怎么知道当前的应用程序究竟
由哪些组件构成？



Android的组件模型

- 一旦确定了应用程序中需要的组件，就应该列在AndroidManifest.xml中

AndroidManifest.xml是一个XML配置文件，它用于定义应用程序中需要的组件、组件的功能及必要条件等。

AndroidManifest.xml结构

```
<?xmlversion="1.0"encoding="utf-8"?>  
<manifest>  
  <application>  
    <activity>  
      <intent-filter>  
        <action/>  
        <category/>  
      </intent-filter>  
    </activity>  
    .....  
    <service>  
      <intent-filter></intent-filter>  
      <meta-data/>  
    </service>  
  </application>  
</manifest>
```

这个文件是必须的！



Android的组件模型：Activity

Activity 就是一个界面（例如Task Manager的界面、Root Explorer 的界面等），界面里面可以放置各种控件

Activity 是为用户操作而提供的可视化用户界面

- ◆ 例如，一个 activity 可以展示一个菜单项列表供用户选择，或者显示一些包含说明的照片
- ◆ 一个短消息应用程序可以包括一个用于显示作为发送对象联系人的列表的 activity、一个给选定的联系人写短信的 activity 以及翻阅以前的短信和改变设置的 activity

尽管它们一起组成了一个内聚的用户界面，但其中每个 activity 都保持独立，每个都是以Activity类为基类的子类实现



Android的组件模型：Activity

一个应用程序可以只有一个 activity，或者如短信应用程序那样包含多个

- activity的数目及其每个activity的作用以取决于应用程序及其设计
- 一般情况下，总有一个应用程序被标记为用户在应用程序启动的时候第一个看到的，从一个 activity 转向另一个的方式是靠当前的 activity 启动下一个

调用Android专有类 Intent 进行屏幕之间的切换。



Android的组件模型：Service

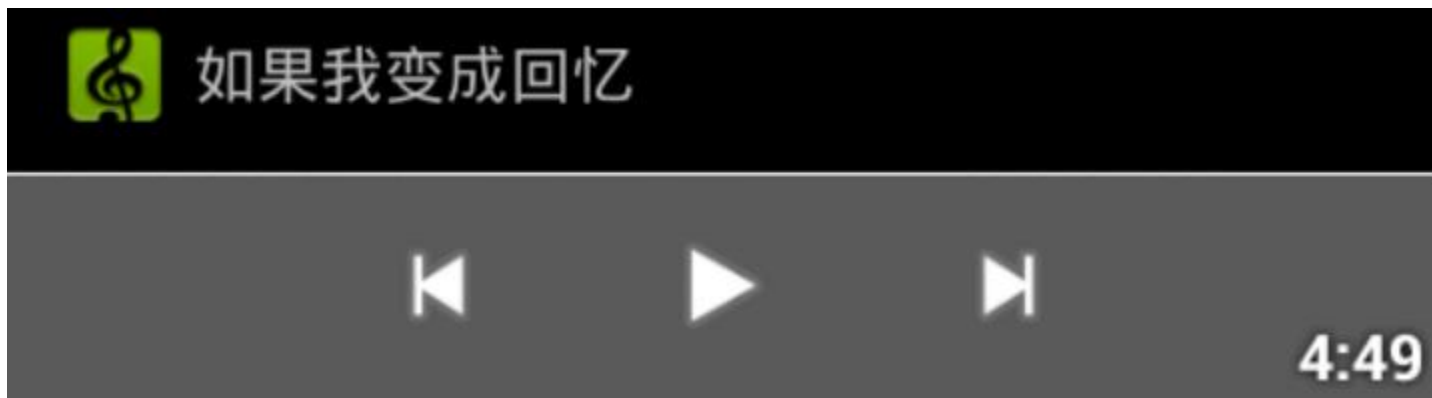
服务是运行在后台的功能模块，如文件下载、音乐播放程序等

- 服务没有可视化的用户界面，而是在一段时间内在后台运行
例如，一个服务可以在用户做其它事情的时候在后台播放背景音乐、从网络上获取一些数据或者计算一些东西并提供给需要这个运算结果的 activity 使用。每个服务都继承自 Service 基类。



Android的组件模型：Service

- 可以连接至（绑定）一个正在运行的服务（如果服务没有运行，则启动之）。连接之后，可以通过那个服务暴露出来的接口与服务进行通信。
- 对于音乐服务来说，这个接口可以允许用户暂停、回退、停止以及重新开始播放。





Android的组件模型： **Content Provider**

- 它是 Android 平台应用程序间数据共享的一种标准接口，它以类似于 URI（Universal Resources Identification）的方式来表示数据，如：
`content://contacts/people/1101`
- 内容提供者将一些特定的应用程序数据供给其他应用程序使用。数据可以存储于文件系统、SQLite 数据库或其它方式



Android的组件模型: **Broadcast Receiver**

Broadcast Receiver 是一个专注于接收广播通知信息，并做出对应处理的组件

- 很多广播是源自于系统代码的。例如，通知时区改变、电池电量低、拍摄了一张照片或者用户改变了语言选项
- 应用程序也可以进行广播，例如通知其他应用程序一些数据下载完成并处于可用状态



Android的组件模型: **Broadcast Receiver**

- Broadcast Receiver 没有用户界面。然而，它们可以启动一个 activity 来响应收到的信息，或者用 NotificationManager来通知用户
- 通知可以用很多种方式来吸引用户的注意力——闪动背灯、震动、播放声音等等。一般来说是在状态栏上放一个持久的图标，用户可以打开它并获取消息。



Android的组件模型： **Broadcast Receiver**

- 与此组件相关的概念是 Intent，Intent 是一个对动作和行为的抽象描述，负责组件之间和程序之间进行消息传递
- Broadcast Receiver 组件提供了一种把 Intent 作为一个消息广播出去、由所有对其感兴趣的程序对其作出反应的机制

例如，为了实现一个系统启动后播放音乐的功能，则可以定义 Intent 为 `android.intent.action.BOOT_COMPLETED`，由 Broadcast Receiver 组件将其进行广播，而系统中的 Media Player 接收到该信息后则进行播放。



Android应用安全



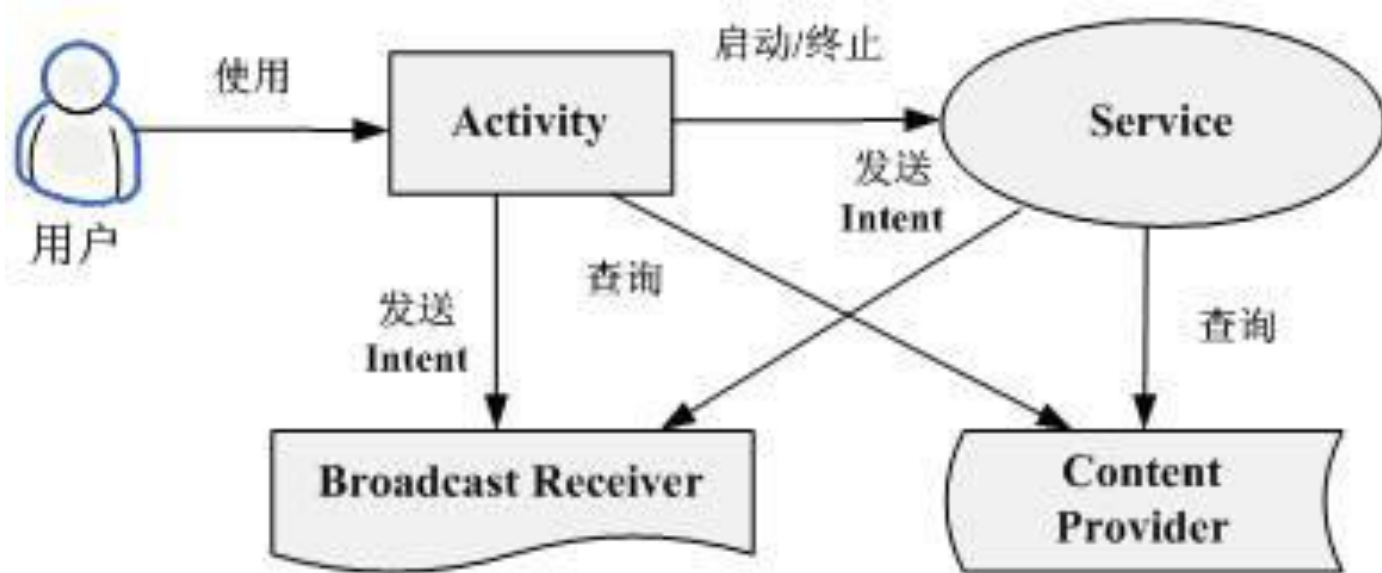
检测应用是否存在
恶意行为进行



研究应用中可能
存在的安全漏洞

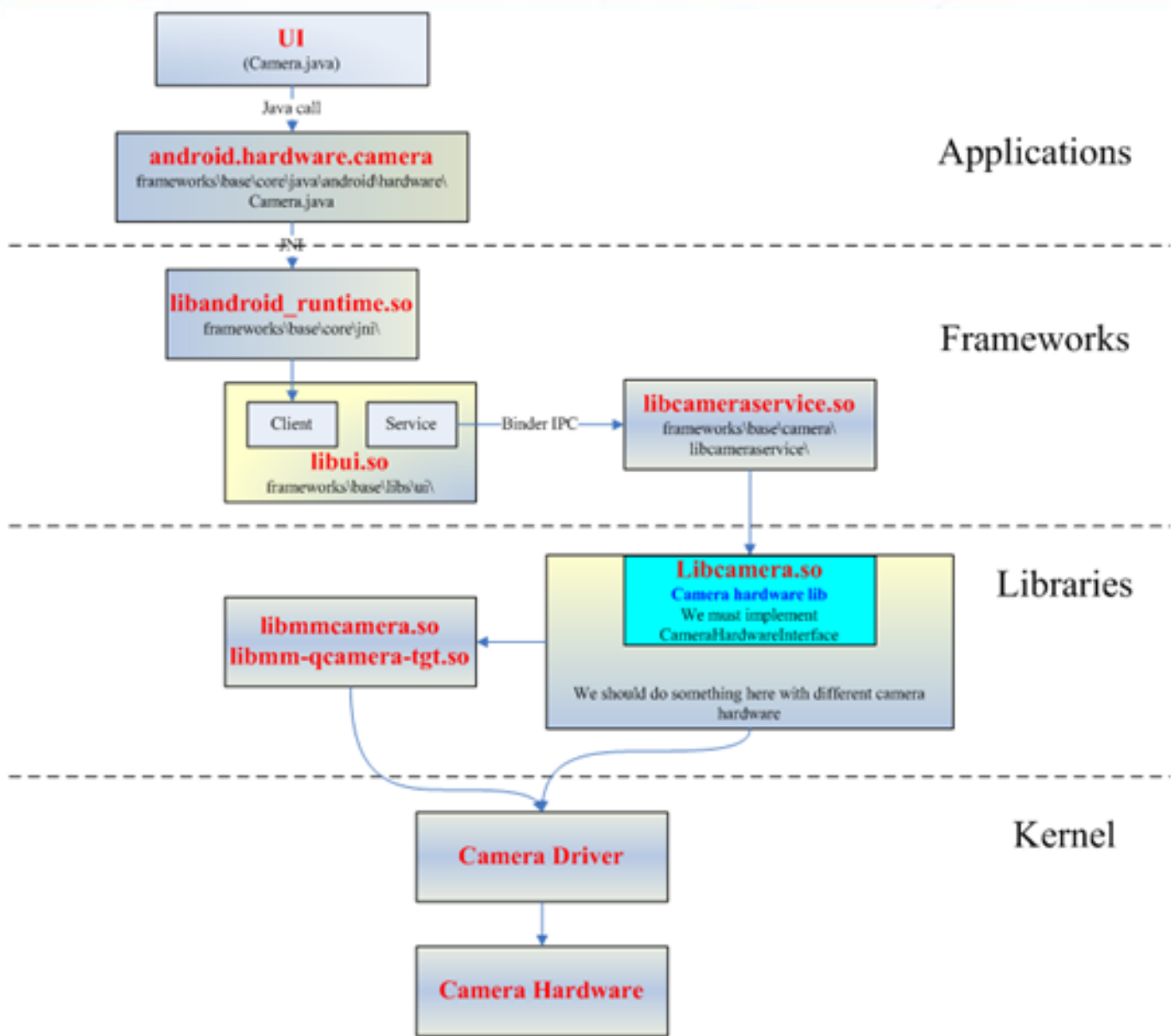


Android的组件模型：四种组件的关系





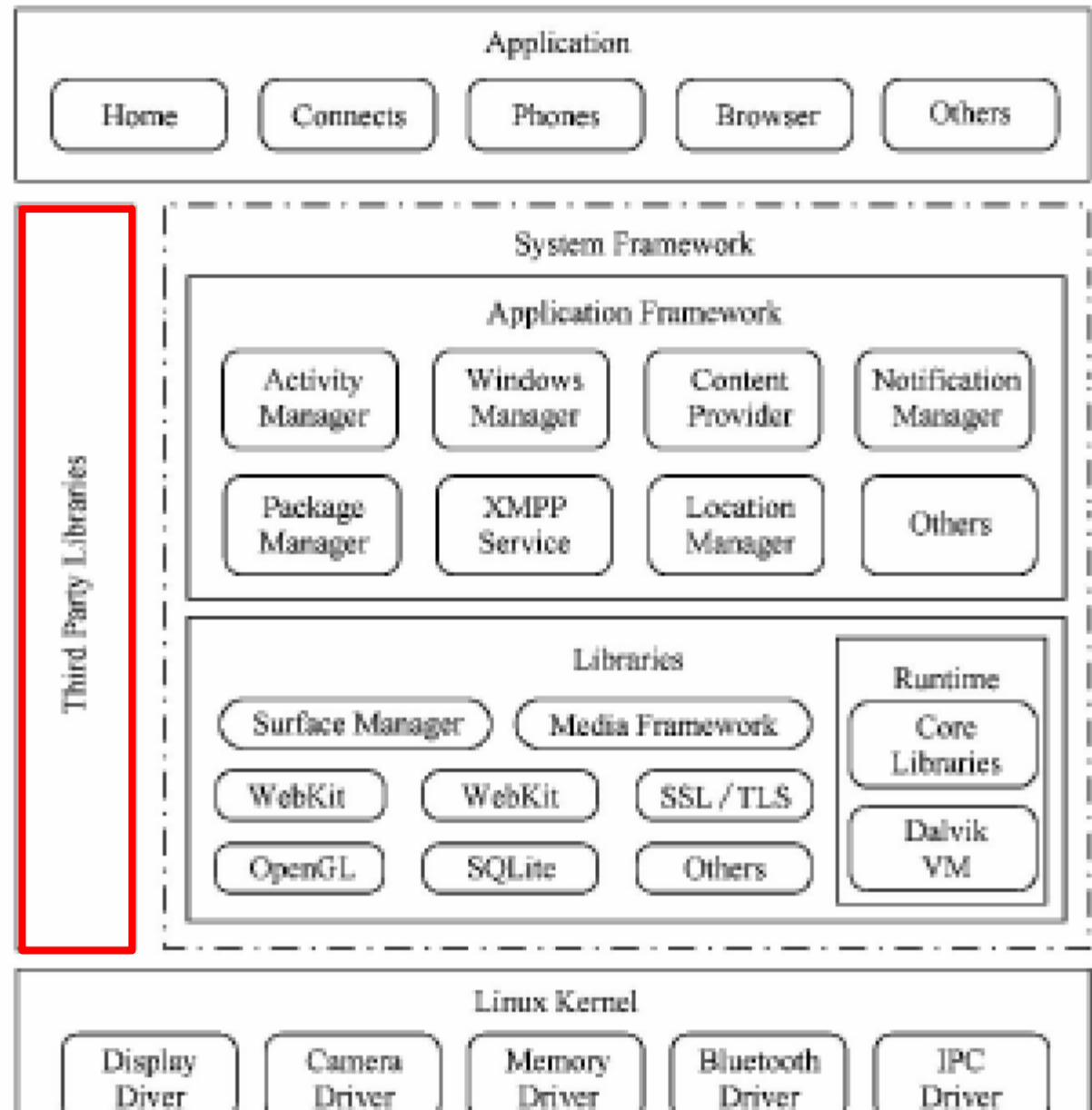
Android架构举例：Camera





Android Architecture: a new trend

第三方类库是指来自第三方平台的API接口集合，开发者可以在自己的应用中使用这些接口来方便地实现官方API未实现的功能。例如，Facebook提供了一个开源的Java库，从而可以让开发者在自己的应用中嵌入Facebook的部分社交功能；Admob是全球最大的移动广告网络，为开发者提供了方便的广告接口。第三方类库独立于Android系统架构实现，但与系统架构处于相同的地位，即使用内核层提供服务，实现、封装功能模块，供应用层调用。





Android或者MIUI为什么卡顿？

优先级的设置

iOS是苹果专门为触控手机做的系统，系统内把touch触控，也就是用户操作响应层放在了最优先的级别，其次再是media 媒体层、service 服务层以及core 架构层。

MIUI是根据Android深度定制的，其优先级响应层级是Android，无法改变，具体来说，优先级别依次是应用、框架、图像处理和架构。与动画相关的图像处理被放到了第三优先级。

以打开手机相册为例，iOS系统中在相册APP没有完全打开时，就可以直接预览到相册的照片，也可以在打开应用的中途就关闭它，体验起来就是跟手且流畅。但在MIUI等安卓系统中，系统必须完全响应相册打开这个动作后，才会响应接下来的动作，体验下来，就是不丝滑、不跟手。



Android或者Android-like卡顿？

系统的优化程度不同

iOS系统是苹果自研的，在研发与后续更新中进行了大量的细节优化，比如iOS7之后，苹果在动画中引入了非线性无缝运动，在滑动应用时将动能保留。

谷歌给予的安卓原生开源代码中只有基础骨骼框架，小米需要根据自身需要定制MIUI系统，这就需要强大的软件团队和对安卓原生开源代码的掌控程度，开发过程中难免翻车。

比如MIUI12系统开发出了更好的动画效果，却出现了大量系统bug，稳定性被拖累。

2021年4月份，B站UP主“木羽说科技”发布一段《小米高端路的最大障碍——MIUI》的视频，来吐槽MIUI12，“息屏按指纹，动不动就闪屏，跳来跳去，这个问题从小米搭载屏幕指纹开始就没解决过”，灵魂质疑让网友感同身受，引来数千万的播放量。



Android或者Android-like卡顿?

iOS系统封闭性带来的优势

iOS系统对于APP的开发框架有极为严格的限制，一定会确保APP与系统的兼容与稳定。而安卓系统上的APP，是软件开发者面对不同安卓系统、不同机型版本开发的通用的APP，很难保证对MIUI系统，对小米的各种机型完美兼容，难免会出现闪退、卡顿等各种bug。

Android
安全概述



Android Security Objectives

Android seeks to be the **most secure and usable** OS for mobile platforms by re-purposing traditional OS security controls to:



Protect user data



Protect system resources
(including the network)



Provide application isolation



key security features

To achieve these objectives, Android provides key security features

Robust security at the OS level through the Linux kernel

Mandatory application sandbox for all applications

Secure interprocess communication

Application signing

Application-defined and user-granted permissions



Linux Security

The foundation of the Android platform is the Linux kernel. The Linux kernel itself has been in widespread use for years, and is used in millions of security-sensitive environments. Through its history of constantly being researched, attacked, and fixed by thousands of developers, Linux has become a stable and secure kernel trusted by many corporations and security professionals.



Linux Security

As the base for a mobile computing environment, the Linux kernel provides Android with several key security features, including:

- A user-based permissions model
- Process isolation
- Extensible mechanism for secure IPC
- The ability to remove unnecessary and potentially insecure parts of the kernel



Linux Security

As a multiuser operating system, a fundamental security objective of the Linux kernel is to isolate user resources from one another. The Linux security philosophy is to protect user resources from one another.

Thus, Linux:

- Prevents user A from reading user B's files
- Ensures that user A does not exhaust user B's memory
- Ensures that user A does not exhaust user B's CPU resources
- Ensures that user A does not exhaust user B's devices (e.g. telephony, GPS, Bluetooth, etc.)

“井水不犯河水”





The Application Sandbox

Each Android package (.apk) file installed on the device is given its own unique Linux user ID, creating a sandbox for it and preventing it from touching other applications (or other applications from touching it).

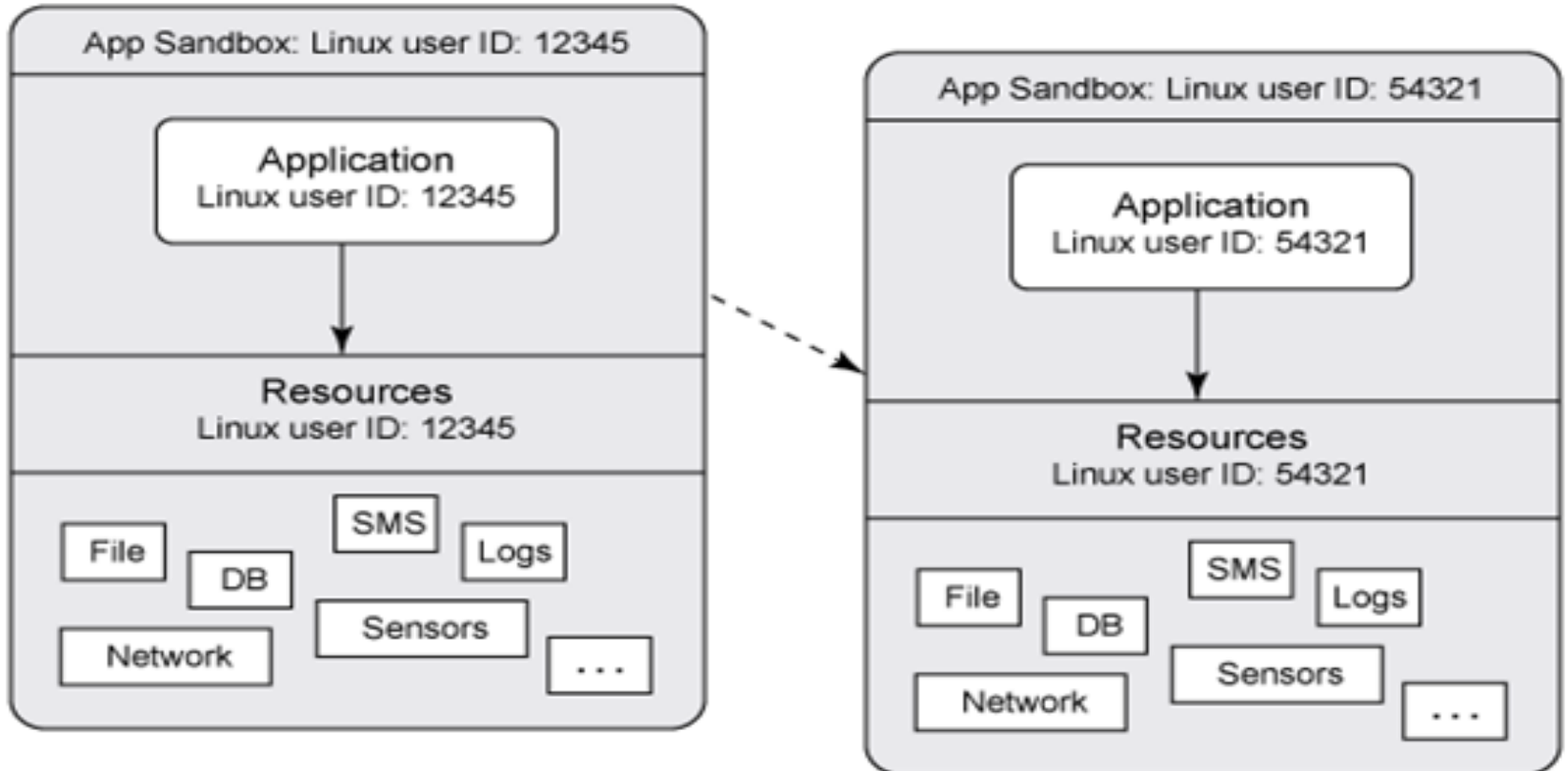
USER	PID	PPID	VSIZE	RSS	WCHAN	PC	NAME
app_16	2855	2363	216196	20960	ffffffff	afd0ee48	S com.android.providers.calendar
app_91	4178	2363	218872	25076	ffffffff	afd0ee48	S jackpal.androidterm

This user ID is assigned to it when the application is installed on the device, and remains constant for the duration of its life on that device.



The Application Sandbox

Android application/process space



Two applications on different processes (with different user-ids)

<http://CEnriqueOrtiz.com>



The Application Sandbox

Android sandbox 通过利用开源工具动态分析、静态分析Android的相关应用，发现应用的具体行为，从而进行判断Android应用的危险程度

- Droidbox
通过hook系统API对apk程序进行监控
- Apimonitor
- AndroGuard
主要应用于Android应用的静态分析



The Application Sandbox

Since the Application Sandbox is in the kernel, this security model extends to native code and to operating system applications.

- All of the software above the kernel, including operating system libraries, application framework, application runtime, and all applications run within the Application Sandbox.





Interprocess Communication

Processes can communicate using any of the traditional UNIX-type mechanisms. Examples include the file system, local sockets, or signals. However, the Linux permissions still apply.



Interprocess Communication

Android also provides new IPC mechanisms:

- **Binder:** A lightweight capability-based remote procedure call mechanism designed for high performance when performing in-process and cross-process calls. Binder is implemented using a custom Linux driver.

See <https://developer.android.com/reference/android/os/Binder.html>.

- **Services:** Services (discussed above) can provide interfaces directly accessible using binder.



Application Signing

Every application that is run on the Android platform must be signed by the developer. Applications that attempt to install without being signed will be rejected by either Google Play or the package installer on the Android device.

Signing

New	
Go Into	
Open in New Window	
Open Type Hierarchy	F4
Show In	Alt+Shift+W
Copy	Ctrl+C
Copy Qualified Name	
Paste	Ctrl+V
Delete	Delete
Remove from Context	Ctrl+Alt+Shift+Down
Build Path	
Source	Alt+Shift+S
Refactor	Alt+Shift+T
Import...	
Export...	
Refresh	F5
Close Project	
Close Unrelated Projects	
Assign Working Sets...	
Run As	
Debug As	
Profile As	
Validate	
Team	
Compare With	
Restore from Local History...	
Android Tools	
Configure	
Properties	Alt+Enter

```
D:\>keytool -genkey -v -keystore liufeng.keystore -alias liufeng.keystore -keyalg RSA -validity 20000
```

输入keystore密码:

再次输入新密码:

您的名字与姓氏是什么?

[Unknown]: liufeng

您的组织单位名称是什么?

[Unknown]: secret

您的组织名称是什么?

[Unknown]: secret

您所在的城市或区域名称是什么?

[Unknown]: guiyang

您所在的州或省份名称是什么?

[Unknown]: guizhou

该单位的两字母国家代码是什么

[Unknown]: CN

CN=liufeng, OU=secret, O=secret, L=guiyang, ST=guizhou, C=CN 正确吗?

[否]: y

正在为以下对象生成 1,024 位 RSA 密钥对和自签名证书 (SHA1withRSA) (有效期为 20,000 天):

CN=liufeng, OU=secret, O=secret, L=guiyang, ST=guizhou, C=CN

输入<liufeng.keystore>的主密码

(如果和 keystore 密码相同, 按回车):

[正在存储 liufeng.keystore]



www.Linuxidc.com



Application Signing

On Google Play, application signing bridges the trust Google has with the developer and the trust the developer has with their application. Developers know their application is provided, unmodified to the Android device; and developers can be held accountable for behavior of their application.



Google Play 前名为 Android Market，是一个由 Google 为 Android 设备开发的在线应用程序商店。一个名为 Play Store 的应用程序会预载在允许使用 Google Play 的手机上，可以让用户去浏览、下载及购买在 Google Play 上的第三方应用程序。2012 年 3 月 7 日，Android Market 服务与 Google Music、Google 图书、Google Play Movie 集成，并将其更名为 Google Play。



Application Signing

Applications can be signed by a third-party (OEM, operator, alternative market) or self-signed. Android provides code signing using self-signed certificates that developers can generate without external assistance or permission. Applications do not have to be signed by a central authority. Android currently does not perform CA verification for application certificates.



Application Verification

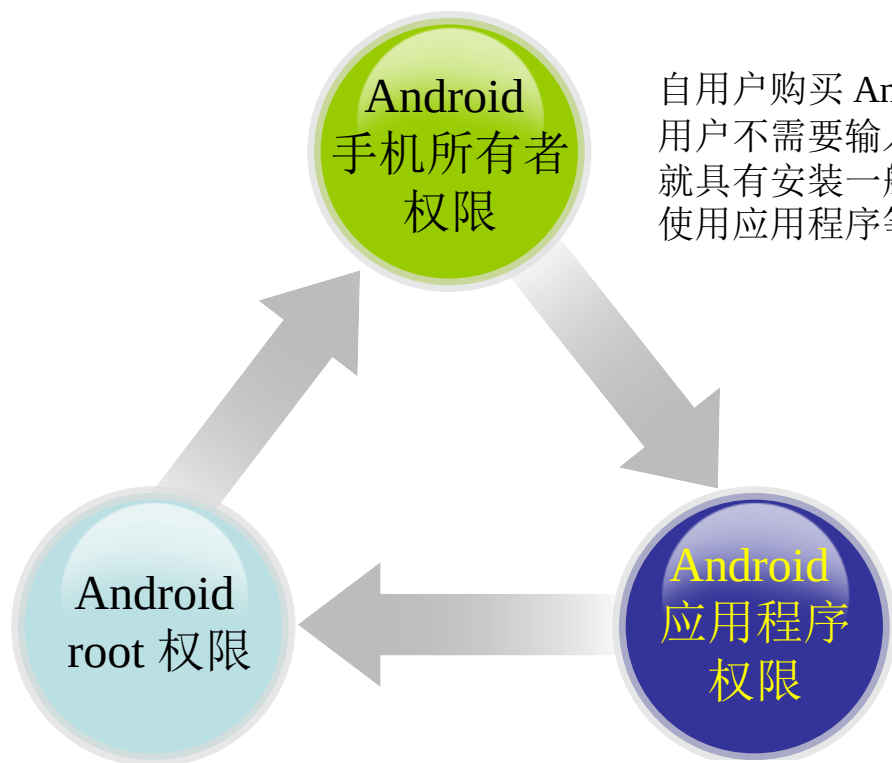
Android 4.2 and later support application verification. Users can choose to enable "Verify Apps" and have applications evaluated by an application verifier prior to installation. App verification can alert the user if they try to install an app that might be harmful; if an application is especially bad, it can block installation.



Android
权限管理

Android 的权限分类

根据用户的使用过程体验，可以将 Android 涉及的权限大致分为如下三类：



自用户购买 Android 手机后，用户不需要输入任何密码，就具有安装一般应用软件、使用应用程序等的权限。

该权限为 Android 系统的最高权限，可以对所有系统中文件、数据进行任意操作。

应用程序对 Android 系统资源的访问需要有相应的访问权限。如：没有获取 Android root 权限的手机无法运行 Root Explorer，因为运行该应用程序需要 Android root 权限。

Android 系统权限定义

Android 系统在/system/core/private/android_filesystem_config.h头文件中对Android 用户/用户组作了如下定义，且权限均基于该用户/用户组设置

```
#define AID_ROOT          0 /* traditional unix root user */
#define AID_SYSTEM        1000 /* system server */
#define AID_RADIO          1001 /* telephony subsystem, RIL */
#define AID_BLUETOOTH      1002 /* bluetooth subsystem */
#define AID_GRAPHICS       1003 /* graphics devices */
#define AID_INPUT          1004 /* input devices */
#define AID_AUDIO          1005 /* audio devices */
#define AID_CAMERA         1006 /* camera devices */
#define AID_LOG             1007 /* log devices */
#define AID_COMPASS        1008 /* compass device */
#define AID_MOUNT          1009 /* mountd socket */
#define AID_WIFI           1010 /* wifi subsystem */
#define AID_ADB            1011 /* android debug bridge (adb) */
#define AID_INSTALL        1012 /* group for installing packages */
#define AID_MEDIA          1013 /* mediaserver process */
#define AID_DHCP           1014 /* dhcp client */
#define AID_SD_CARD_RW     1015 /* external storage write access */
#define AID_VPN            1016 /* vpn system */
#define AID_KEYSTORE       1017 /* keystore subsystem */
#define AID_USB            1018 /* USB devices */
#define AID_DRM            1019 /* DRM server */
#define AID_DRM_IO        1020 /* DRM IO server */
#define AID_GPS            1021 /* GPS daemon */
#define AID_NFC            1022 /* nfc subsystem */

#define AID_SHELL          2000 /* adb and debug shell user */
#define AID_CACHE          2001 /* cache access */
#define AID_DIAG           2002 /* access to diagnostic resources */

/* The 3000 series are intended for use as supplemental group id's only.
 * They indicate special Android capabilities that the kernel is aware of. */
#define AID_NET_BT_ADMIN  3001 /* bluetooth: create any socket */
#define AID_NET_BT        3002 /* bluetooth: create sco, rfcomm or l2cap sockets */
#define AID_INET          3003 /* can create AF_INET and AF_INET6 sockets */
#define AID_NET_RAW       3004 /* can create raw INET sockets */
#define AID_NET_ADMIN     3005 /* can configure interfaces and routing tables. */

#define AID_MISC           9998 /* access to misc storage */
#define AID_NOBODY        9999

#define AID_APP            10000 /* first app user */
```

Android 系统权限定义

值得注意的是：每个应用程序在安装到 Android 系统后，系统都会为其分配一个用户ID，如 app_4、app_11 等

例如，以下是 **Calendar** 和 **Terminal** 软件在 **Android** 系统中进程浏览的结果（其中，黑色字体标明的即为应用分配的用户 ID）：

USER	PID	PPID	VSIZ	RSS	WCHAN	PC	NAME
app_16	2855	2363	216196	20960	ffffffff	afd0ee48	S com.android.providers.calendar
app_91	4178	2363	218872	25076	ffffffff	afd0ee48	S jackpal.androidterm

Android 系统权限定义

在 Android 系统中，用户 / 用户组对文件的访问遵循 Linux 系统的访问控制原则，即根据长度为 10 个字符的权限控制符来决定用户 / 用户组对文件的访问权限。该控制符的格式遵循下列规则：

- 第 1 个字符：表示一种特殊的文件类型。其中字符可为 d（表示该文件是一个目录）、b（表示该文件是一个系统设备，使用块输入/输出与外界交互，通常为一个磁盘）、c（表示该文件是一个系统设备，使用连续的字符输入/输出与外界交互，如串口和声音设备），“.”表示该文件是一个普通文件，没有特殊属性。
- 第 2 ~ 4 个字符：用来确定文件的用户（user）权限；
- 第 5 ~ 7 个字符：用来确定文件的组（group）权限；
- 第 8 ~ 10 个字符：用来确定文件的其它用户（other user，既不是文件所有者，也不是组成员的用户）的权限。
- 第 2、5、8 个字符是用来控制文件的读权限的，该位字符为 r 表示允许用户、组成员或其它人可从该文件中读取数据。短线“-”则表示不允许该成员读取数据。
- 第 3、6、9 位的字符控制文件的写权限，该位若为 w 表示允许写，若为“-”表示不允许写。
- 第 4、7、10 位的字符用来控制文件的执行权限，该位若为 x 表示允许执行，若为“-”表示不允许执行。

Android 应用程序权限申请

- 每个应用程序的 APK 包里面都包含有一个 `AndroidMainifest.xml` 文件，该文件除了罗列应用程序运行时库、运行依赖关系等之外，还会详细地罗列出该应用程序所需的系统访问
- 程序员在进行应用软件开发时，需要通过设置该文件的 `uses-permission` 字段来显式地向 Android 系统申请访问权限

Android
新版本变化

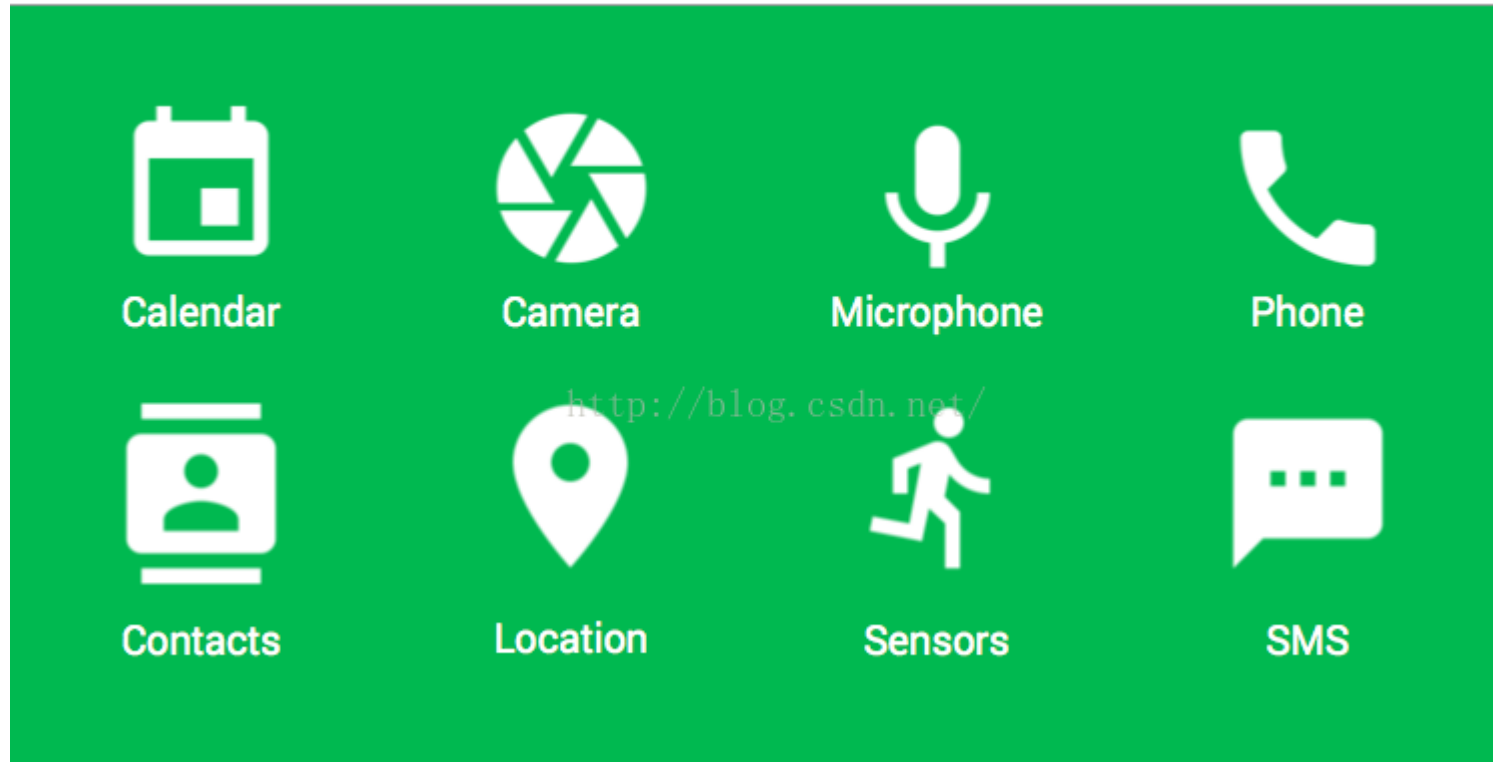
Android M



2015年5月28日，Google I/O大会上正式推出Android M（6.0）。北京时间8月18日早间消息，谷歌周一宣布，Android M中的“M”是单词“Marshmallow”（棉花糖）的缩写。

此前，Android M的“M”引起了种种猜测，其中包括Milkshake（奶昔）、Moonpie（月饼）、Mocha（摩卡咖啡）、Madeleine（玛德琳蛋糕）、Macaroon（马卡龙），以及M&M巧克力豆等。

Android M



在Android-M中，如果是同属于一类的权限，就会被整理到一个组别之下，这个组别将会在用户发出确切请求时来申请获得具体的权限，所以举例来说，如果要获得“读取日历”(READ_CALENDAR)的权限，那用户就会被提示是否授予“日历”权限。

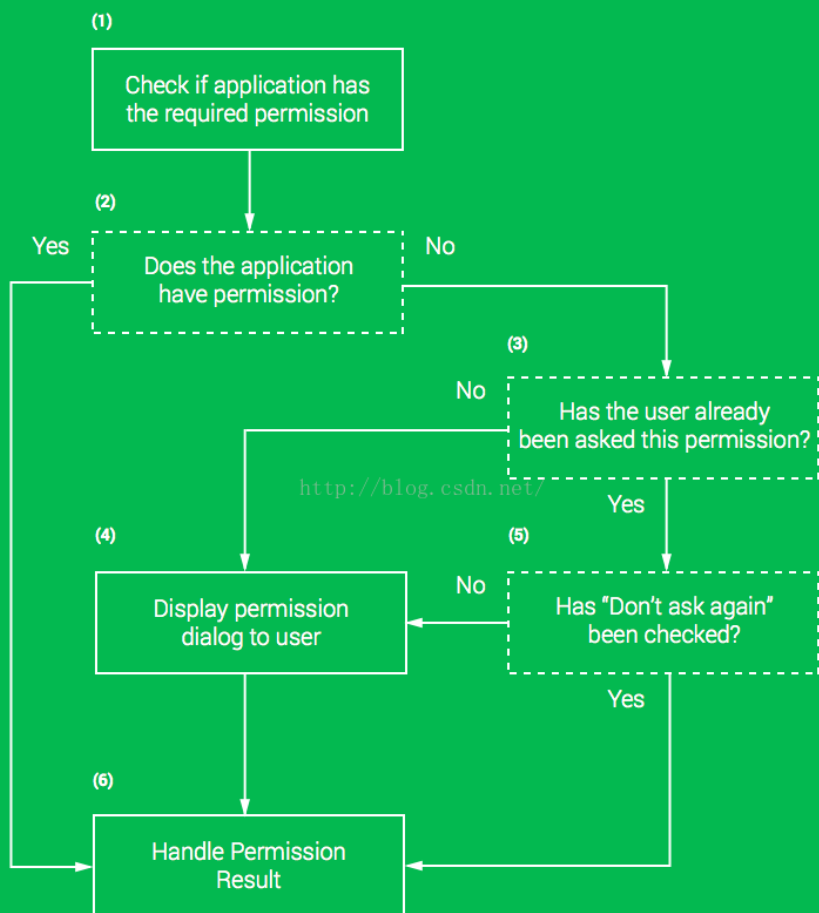
Android M

这些权限组别中，很多都包含了好几个具体权限，可表现为以下分组：

Calendar	<code>android.permission.READ_CALENDAR</code> <code>android.permission.WRITE_CALENDAR</code>
Camera	<code>android.permission.CAMERA</code>
Contacts	<code>android.permission.READ_CONTACTS</code> <code>android.permission.WRITE_CONTACTS</code> <code>android.permission.READ_PROFILE</code> <code>android.permission.WRITE_PROFILE</code>
Location	<code>android.permission.ACCESS_FINE_LOCATION</code> <code>android.permission.ACCESS_COARSE_LOCATION</code>
Microphone	<code>android.permission.RECORD_AUDIO</code>
Phone	<code>android.permission.READ_PHONE_STATE</code> <code>android.permission.CALL_PHONE</code> <code>android.permission.READ_CALL_LOG</code> <code>android.permission.WRITE_CALL_LOG</code> <code>com.android.voicemail.permission.ADD_VOICEMAIL</code> <code>android.permission.USE_SIP</code> <code>android.permission.PROCESS_OUTGOING_CALLS</code>
Sensors	<code>android.permission.BODY_SENSORS</code> <code>android.permission.USE_FINGERPRINT</code>
SMS	<code>android.permission.SEND_SMS</code> <code>android.permission.RECEIVE_SMS</code> <code>android.permission.READ_SMS</code> <code>android.permission.RECEIVE_WAP_PUSH</code> <code>android.permission.RECEIVE_MMS</code> <code>android.permission.READ_CELL_BROADCASTS</code>
Sensors	<code>android.permission.READ_EXTERNAL_STORAGE</code> <code>android.permission.WRITE_EXTERNAL_STORAGE</code>

运行时权限

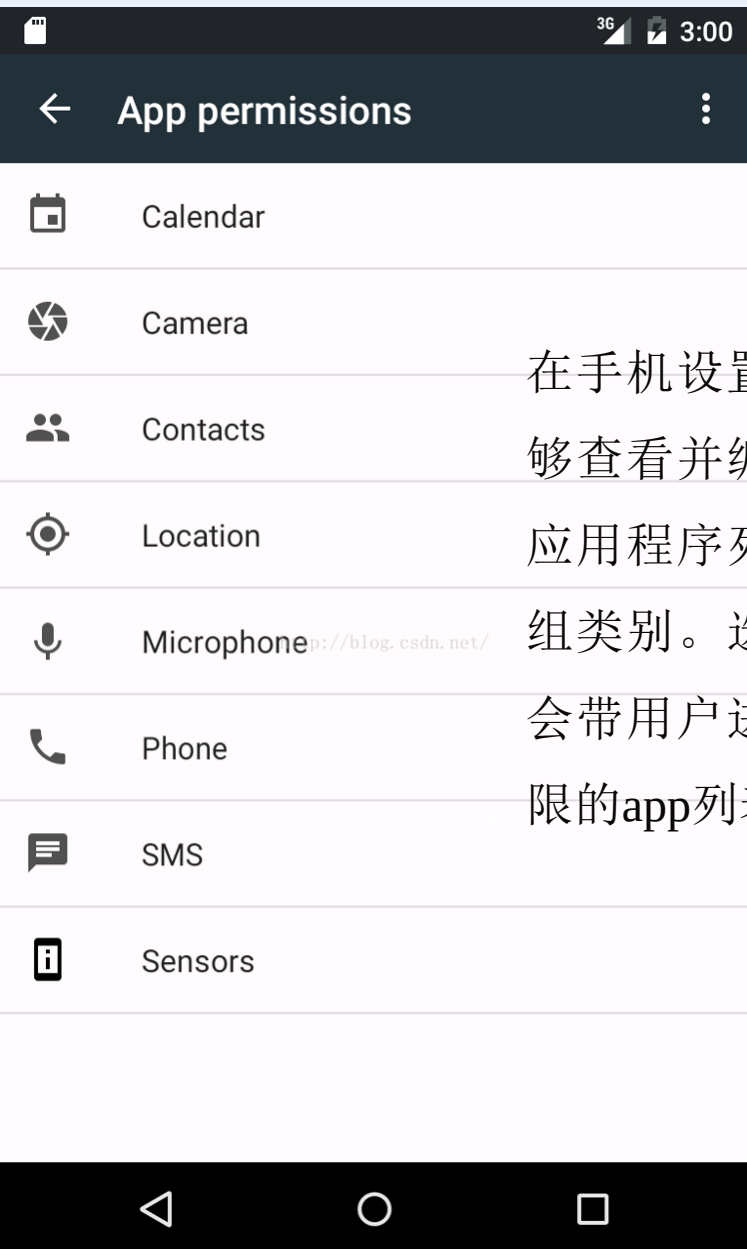
在安装程序之前首先需要接受一大串的权限许可。最新的权限授予将不再做这样的要求，所以在Google Play商店里打算安装某个app的时候，点击安装按钮就会是真的安装应用程序，而不是不得不先去授予这样那样的权限。



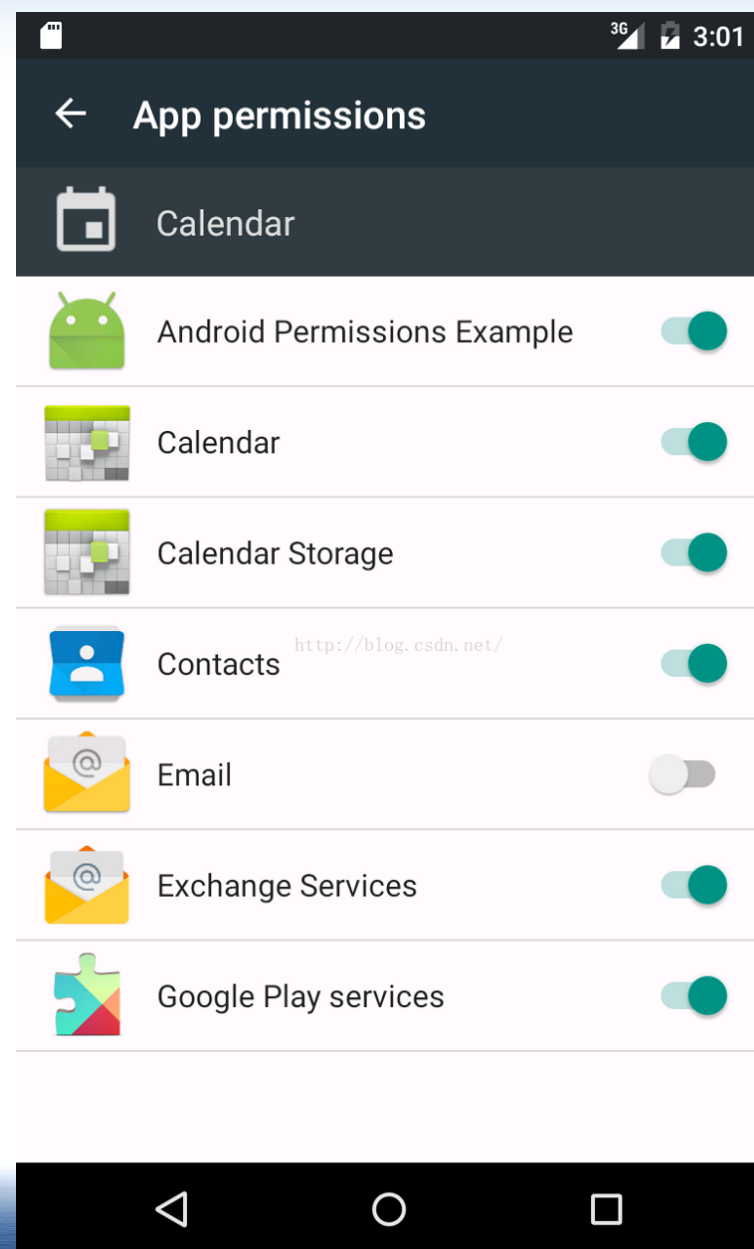
在应用运行过程中，有些点击会触发提示获取权限来使用某些具体的功能，在这一点上，用户需要检查自己的系统来确认自己是不是拥有这样的权限。

图：系统将会在授予用户一个可以继续运行下去的路径之前，做好一系列的检查。

M版的应用程序权限



在手机设置里面，用户能够查看并编辑给定权限的应用程序列表——权限分组类别。选择其中一个将会带用户进入对应给定权限的app列表



Android的最新版本

Google的安卓开发团队在2018年7月发布了Android P（也就是Android 9.0）的候选版本。



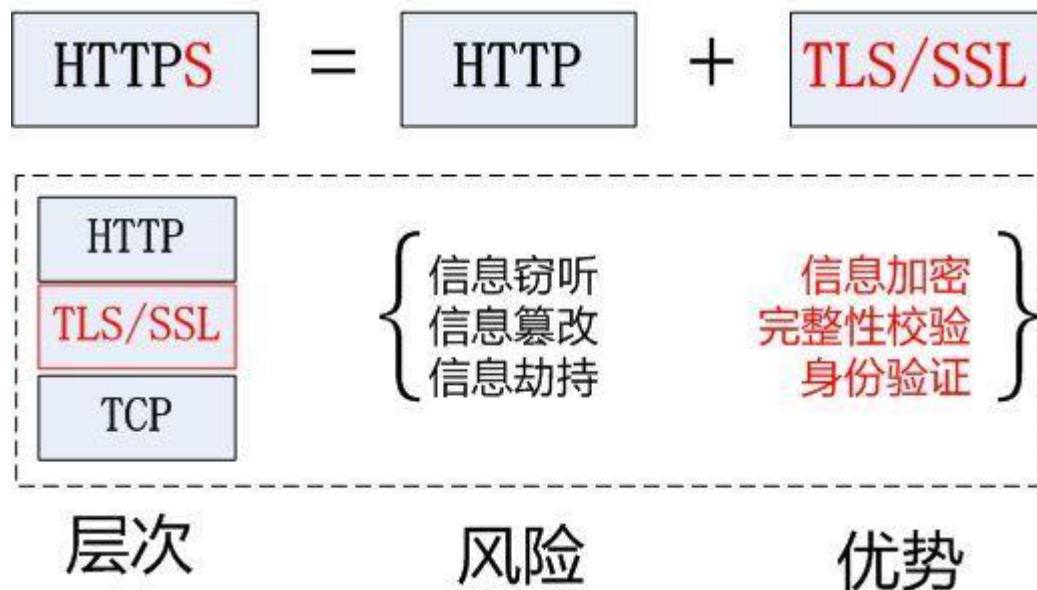
提高指纹验证的可靠性

- Android P为了提高生物认证的可靠性和可信性，增加了基于个体生物特征的身份验证方法和基于改进生物特征的安全模型。
- 而Android 8.1也用了SAR欺骗接受率和IAR伪造接受率两个指标，来辅助身份特征系统尽量减少遭受攻击的可能性。
- 另外，Android P为了让用户觉得更可靠，还提供了指纹验证时标准化的UI布局。APP开发人员调用BiometricPrompt这个应用程序编程接口，就可以调出来指纹验证的功能了。

证书签名机制v3

- Android P 支持的Android安装包签名机制v3与v2版本的不同之处是增加了密钥轮转。密钥轮转机制中有ApkSignerLineage功能，我们可以非常方便地对新证书签名，和Android安装包绑定。
- 在Android P系统中，这个签名机制v3默认开启，但是并不影响我们坚持使用老版本的签名证书。

默认支持安全传输协议



Android P默认支持HTTPS安全传输协议，可以减少用户在数据安全方面的忧虑。

增加保护确认的API

- 设备只要运行Android P，就会有用于保护确认的应用程序编程接口。一些应用程序在用户进行敏感交易或者付账单的时候，可以用ConfirmationPrompt类向使用者弹出一个确认或者是否允许操作的对话框。
- 用户如果点击了确认，应用程序就会收到一个在受信任的执行环境（TEE）生成的加密签名。而这个加密签名是受HMAC保护的。这样可以保护用户输入的数据。

硬件安全

- 硬件安全中StringBox Keymaster的功能模块的更新有益于每一个用户。Android P 使用加密算法的密钥子集来支持StringBox Keymaster功能，这个功能模块可以避免应用程序的数据包被篡改的问题。
- 值得一提的是这个功能模块有CPU、安全存储区和真实随机数生成器。

后台禁止外围设备自动运行

- 在新版本Android P这个系统中，应用程序不能直接访问麦克风、摄像头、传感器这些外围设备。
- 一旦应用程序试图在后台偷偷访问这些组件数据时，会给用户提示。另外还会停止这些外围设备，并立即停止传感器返回数据。



备份数据自动加密

- 以前，备份数据的加密过程在服务器端完成，用户并不能看到。现在 Android P 的策略是，如果用户需要恢复备份文件，就得输入设备PIN码、图形解锁码或者指纹、刷脸这些生物特征。
- 在客户端加密备份的数据，对用户来说显得更安全。毕竟大家都觉得眼见为实嘛。

今天的作业

编写一个**Android**的小程序，要求：

- （1）至少用到四个常用组件中的两个；
- （2）提交完整的project和最终的apk包；
- （3）2022年5月31日17：00之前提交。

Summary of The Class



- Android的系统架构及存在的安全隐患
分层架构、四大组件
- Android的安全机制
Kernel-level vs. Application security
Android的五个安全特性
- Android 新版本及其安全机制